Theoretical Foundations of Artificial Intelligence

Unit V – Structured Representation of Knowledge

1. Semantic Nets & Partitioned Semantic Nets
2. Frames as Sets and Instances
3. Slots as full-fledged objects
4. Property Inheritance through Tangled hierarchies
5. Conceptual Dependency
6. Conceptual Dependency Graphs & Scripts
7. Merits and Demerits of strong Slot-filler structures.

More complex, organized ways to store knowledge than simple logic statements.

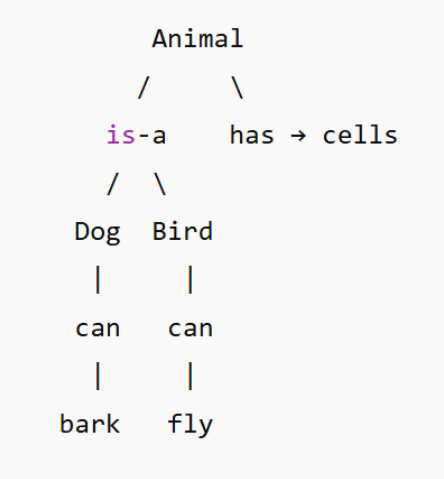
1. Semantic Networks & Partitioned Semantic Networks

What is a Semantic Net?

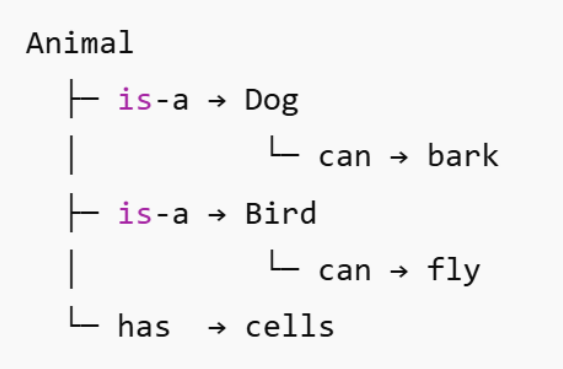
A **semantic net** is a graph-based way of representing knowledge.

- **Nodes** represent concepts or entities (e.g., *Dog*, *Fido*, *Bark*).
- **Edges** (labelled links) represent relations between nodes (e.g., *is-a*, *has-a*, *can*).

Example: Animals and their properties



(OR)



Interpretation (simple):

- **Nodes** = concepts (Animal, Dog, Bird)
- **Links** = relations (is-a, has, can)
- Inheritance: Dog inherits properties of Animal.

Why useful?
This is how humans think: connected ideas.

A semantic net is like a **mind-map**: ideas (nodes) connected by labeled arrows (edges) that say how they relate.

Partitioned Semantic Net

A **partitioned semantic net** organizes the graph into **partitions** or named subgraphs. Partitions let you:

- Group related facts (e.g., facts true in 1990 vs 2020).
- Represent scoped knowledge: defaults, contexts, or hypothetical beliefs.

Imagine 2 different "contexts":

- General world facts
- What *Alice believes*

```
[General Facts Partition]
   Bird → can → fly

[Alice's Belief Partition]
   Penguin → cannot → fly
```

Why?
Partitions avoid conflicts and allow multiple viewpoints.

Example:

```
Partition 1: "Family Relationships"
    John --father-of--> Mary
    John --husband-of--> Lisa

Partition 2: "Work Relationships"
    John --manager-of--> Mary
    John --colleague-of--> Lisa
```

Why Partitioning is Important:

- Prevents relationship conflicts (John is Mary's father AND manager)
- Allows different contexts to coexist
- Reduces complexity in large networks

Example use-cases:

- *Temporal partitioning*: "In 1990, X was true" vs "In 2020, not true."
- *Belief partitioning*: "Alice believes A" in Alice's partition, separate from objective facts.

2. Frames as Sets and Instances

What they are: Templates or blueprints that define the structure for a category of objects.

Structure:

Frame: Template for representing objects/concepts
Slots: Attributes or properties
Fillers: Values for slots

A knowledge representation structure that packages information about a concept in **slots** and **fillers**. Like a blank form (in a bank) or a **class** in programming.

Frame = template (like a class)
Instance = a filled frame (like an object)

Frame Example:

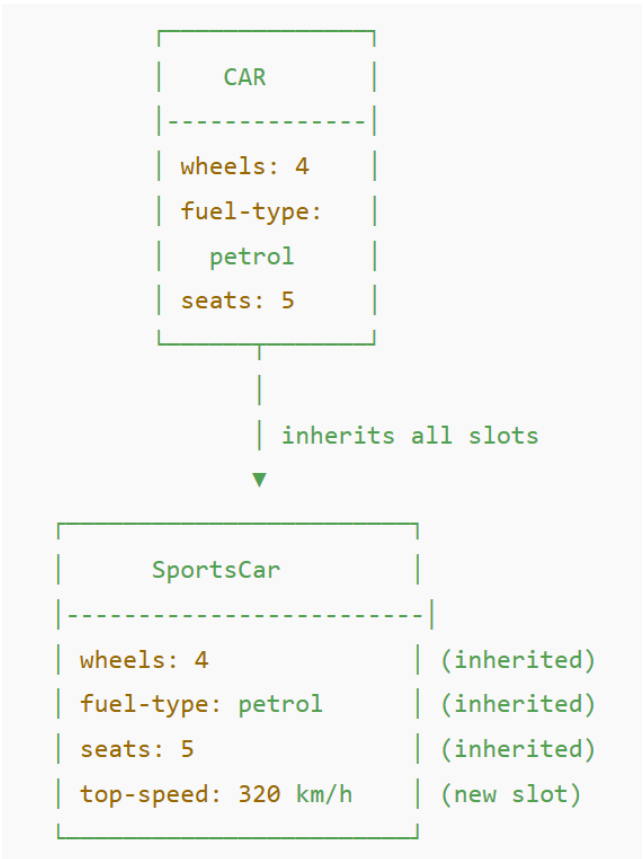
Car Frame:

```
Frame: Car
  slots: make, model, year, color, engine_type, has_sunroof?
```

Instance:

```
Car1:
  make = Toyota
  model = Corolla
  year = 2018
  color = Blue
  has_sunroof? = No
```

Inheritance: Frames are organized in a hierarchy. A SportsCar frame inherits all slots from the CAR frame and can add its own (e.g., top-speed).



Explanation (simple):

- **SportsCar** inherits **all slots from CAR** (wheels, fuel-type, seats).
- It adds its own extra slot: **top-speed**.
- This is exactly how inheritance works in frames.

Frame Features:

- **Inheritance**: Child frames inherit from parents
- **Demons**: Procedures triggered by slot access
- **Multiple inheritance**: Frame can have multiple parents

3. Slots as Full Fledged Objects

In richer frame systems, **slots** themselves are not just name→value pairs. Each slot can be a **full object**.

- **Default values**
- **Constraints** (type, allowed values)
- **Procedures** (called *procedural attachments* or *demons*) executed when the slot is read or written
- **Multiple values**, priorities, provenance (where the info came from)

Here's how a **slot itself** can be smart:

```
Slot: age
  type: integer
  allowed_range: 0-120
  default: compute from birth_date
  on-change: recompute_senior_citizen_status()
```

A slot is like a **smart field** in an online form:

- Auto-fills
- Validates
- Triggers actions

Traditional Slots Vs Full-Fledged Slots

Traditional:

```
Slot: Age
Value: 25
```

Full-Fledged:

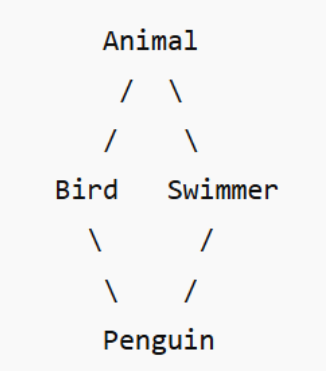
```
Slot: Age
Properties:
  - DataType: Integer
  - Range: 0-150
  - DefaultValue: 0
  - Validation: mustBePositive
Methods:
  - validate(): checks if value is within range
  - increment(): increases age by 1
  - getCategory(): returns "Child", "Adult", "Senior"
Constraints:
  - If Age < 18 then CanDrive = false
```

4. Property Inheritance

What is inheritance here?
Frames (or classes) often inherit slots/values from parent frames:

Inheritance Mechanisms:

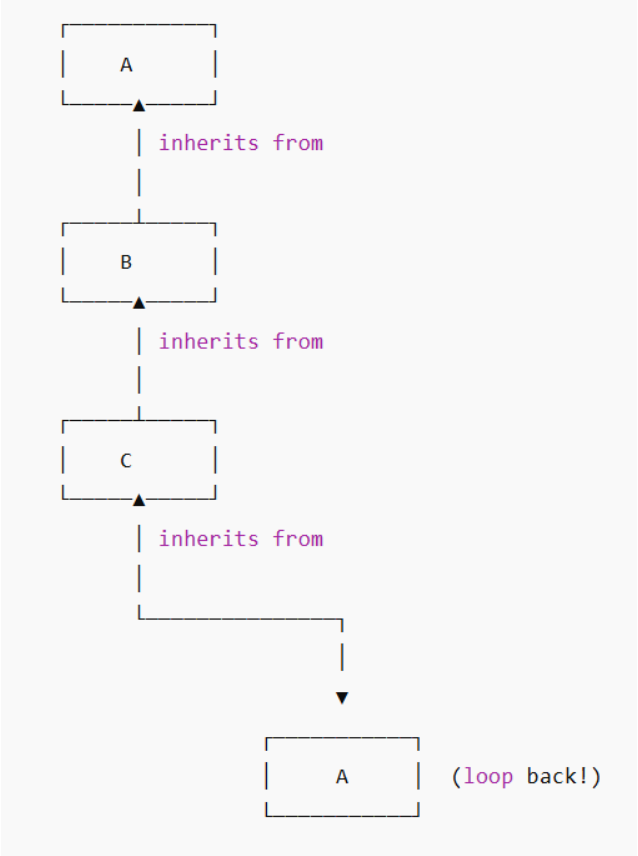
- **Simple Inheritance**: Single parent chain
- **Multiple Inheritance**: Multiple parent chains
- **Exception Handling**: Override inherited properties (Inheritance gives defaults, but **exceptions happen when a subclass does NOT follow the general rule.**)



- Bird → can-fly = yes
- Swimmer → can-swim = yes
- Penguin inherits both, but:
    - can-fly = **no** (exception)

Main problems:

- Conflicting properties
- Exceptions
- Cycles in inheritance



Imagine:

- Your boss reports to you,
- You report to your manager,
- And the manager reports back to your boss.

Who is actually in charge?

No one can tell — it's a loop.

Tangled Hierarchies (Multiple Inheritance):

An object can inherit from multiple parents and get conflicting properties.

Example:

- Penguin is both Bird and Swimmer.
- Bird → can-fly = true
- Penguin should have can-fly = false (exception).

Problems & solutions:

- **Conflicts:** Two parents give different values for the same slot.
  - *Resolution strategies:* specific overrides, priority ordering, explicit exceptions.
- **Exceptions:** Specific subclasses break a general rule (ostriches, penguins).
- **Inheritance loops / cycles** can complicate resolution.

Exceptions:

Why is this needed?

Because real-world categories have **irregular members**.
Examples:

- Birds usually fly → exception: ostrich, penguin
- Mammals usually don't lay eggs → exception: platypus
- Vehicles normally have wheels → exceptions: boats, helicopters

If inheritance blindly applied parent rules, we would get incorrect conclusions.

So, Exception Handling means: When a subclass has a different property than the parent, the child's value overrides the parent's default.

5. Conceptual Dependency (CD)

Conceptual Dependency is a theory that represents the meaning of natural language sentences in a **language-independent set of primitives.** The aim: capture semantics in a way that different surface sentences with the same meaning map to the same internal representation.

Key idea

- Replace language-specific verbs with primitive actions (like *PTRANS* = physical transfer, *ATRANS* = transfer of abstract possession, *MTRANS* = transfer of information).
- Represent actors, objects, and the primitive action and its parameters.

Primitive Actions:

- PTRANS: Physical transfer
- ATRANS: Abstract transfer
- PROPEL: Apply force : Kicking a Ball
- MOVE: Change location : A body part changes its **position** from one place to another. (Raising your hand)
- GRASP: Grab object: Picking up a cup
- INGEST: Take into body : A person/animal takes something **inside** their body (eat food)
- EXPEL: Remove from body: Something **leaves** the body (Sweating)
- MTRANS: Mental transfer: Transfer of **information** from one mind to another.
- MBUILD: Construct mental information: A person builds new mental knowledge **inside their mind**, not transferred from someone else. (You realize something on your own)
- SPEAK: Produce sounds
- ATTEND: Focus sense organ

Example: "John gave Mary a book"

ATRANS (book, from John, to Mary)

Cause: John
Result: Mary has book

Physical Transfer (PTRANS): Movement of a **physical object** from one place to another.

Example:

- A delivery guy moves a parcel from warehouse to customer.
- You push a chair from one room to another.

Abstract Transfer (ATRANS): Movement of an **intangible thing** — like ownership, permission, knowledge, control, etc.

Example:

- A bank **approves a loan** → transfer of **authorization**.
- A manager **assigns responsibility** → transfer of **duty**.

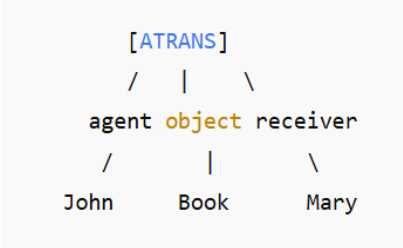
6. Conceptual Dependency Graphs and Scripts

Conceptual Dependency Graphs (CDGs)

A **CDG** is a graph form of conceptual dependency representation:

- Nodes = actions (primitive) and participants (actors, objects).
- Edges = roles or relations (agent, object, instrument, destination).

Example graph for "John gave Mary the book":



Conceptual Dependency Scripts

A **script** is a stereotypical **sequence of events** for a familiar situation. Scripts capture typical order, roles, and expected outcomes.

Restaurant Script Example:

1. Enter restaurant
2. Find table
3. Sit down
4. Order food
5. Eat
6. Pay bill
7. Leave

Why Do We Need Scripts?

Because people **never describe everything**.
Example:
“I went to the restaurant and left a tip.”

You didn’t say you:

- sat down
- read the menu
- ordered food
- ate
- got the bill

…but humans automatically understand all those steps.

Scripts allow AI to understand all *implied* steps.

Example2: DOCTOR VISIT SCRIPT

```
ARRIVAL
  - Patient arrives
  - Registers at desk

CHECKUP
  - Nurse checks vitals
  - Doctor examines patient

TREATMENT
  - Doctor prescribes medicine/tests

LEAVING
  - Patient pays fees
  - Patient leaves clinic
```

How CDGs and Scripts work together

- **CDGs** represent individual actions/events semantically.
- **Scripts** string CDGs into predictable event sequences for everyday situations.

Example:

- CDG = **single movie scene** described in storyboard language.
- Script = the **movie plot** (sequence of scenes) with expectations about what happens next.

Script Components:

- **Entry conditions**: When script starts
- **Roles**: Participants (customer, waiter)
- **Props**: Objects used (menu, food)

- **Scenes**: Sequence of events
- **Results**: End state

Example: We will use the Restaurant Script as the main example

```
ENTRY CONDITIONS:
   - Hungry, restaurant open, money available

ROLES:
   - Customer, Waiter, Chef, Cashier

PROPS:
   - Menu, Food, Table, Bill, Money

SCENES:
   1. Enter and sit
   2. Order food
   3. Eat food
   4. Pay bill and leave

RESULT:
   - Customer full
   - Restaurant gets payment
```

Merits & Demerits of Strong Slot-Filler Structures

Merits of Structures (Nets, Frames): Intuitive (look like real word connections), support inheritance, good for default reasoning (assuming typical facts unless told otherwise), compact (saves memory – because we store common facts once at the parent level).

Demerits: Can become complex for large domains, inference can be complicated, may not be logically formal. Sometime the meaning of the link is ambiguous.
For example: An arrow "has →" could mean possession, property, part-of — it's not always precise.

When to use what

- **Use semantic nets / frames** when you want explicit, human-readable structured knowledge and inheritance.
- **Make slots powerful** if you need dynamic behavior, validation, or provenance.
- **Be careful with tangled hierarchies** — design conflict resolution rules early.
- **Use conceptual dependency and CDGs** for language understanding or when you want language-neutral semantic representations.
- **Use scripts** for situations with strong stereotypical sequences (restaurants, flights, buying).